МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Факультет информатики и вычислительной техники
Кафедра вычислительной техники

Ответы на контрольные вопросы

по дисциплине «Технология разработки программного обеспечения»
Лабораторные работы 3, 6, 7, 8

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ржавин В. В.

Чебоксары, 2026

Содержание

1. Лабораторная работа 3. Моделирование бизнес-процессов в BPMN — 21 вопрос

2. Лабораторная работа 6. Модель анализа — 29 вопросов

   2.1. Контрольные вопросы — 10

   2.2. Контрольные вопросы по UML — 19

3. Лабораторная работа 7. Проектирование веб-API — 12 вопросов

4. Лабораторная работа 8. Документирование веб-API — 15 вопросов

Итого: 77 вопросов и ответов.

1. Лабораторная работа 3. Моделирование бизнес-процессов в BPMN

1. Классификация бизнес-процессов

Основные (операционные) процессы создают ценность для клиента и приносят прибыль (производство, продажи, оказание услуг).

Обеспечивающие (вспомогательные) поддерживают основные процессы (ИТ-поддержка, бухгалтерия, кадры, снабжение).

Управляющие — стратегическое и оперативное планирование, контроль, аудит, корпоративное управление.

Процессы развития направлены на улучшение компании и создание новых продуктов (R&D, модернизация, внедрение новых технологий).

2. Отличие функционального и процессного подходов

Функциональный подход делит организацию на отделы (вертикальное управление); сотрудники отвечают за свою узкую функцию, из-за чего возможны бюрократия и потери информации на стыках подразделений. Процессный подход ориентирован на сквозной результат (горизонтальное управление): процесс проходит через разные отделы и объединяет их общей целью — созданием ценности для клиента.

3. Четыре шага использования BPM-систем (жизненный цикл BPM)

1) Моделирование — описание процесса в виде схемы (AS-IS, TO-BE).

2) Исполнение — запуск процесса в BPM-движке.

3) Контроль — отслеживание метрик, времени выполнения и «узких мест».

4) Улучшение — анализ данных и оптимизация модели процесса.

4. Отличие BPMN от других нотаций

Главное отличие — исполняемость: модель BPMN в формате XML может транслироваться в код и выполняться BPM-движком. Кроме того, BPMN — стандартная нотация, понятная как бизнесу, так и ИТ, с формальной семантикой событий, задач и шлюзов.

5. BPMN отражает процессное моделирование. Как вы это понимаете?

Нотация позволяет наглядно показать сквозной алгоритм работы: с чего процесс начинается (стартовое событие), кто в нём участвует (пулы и дорожки), в какой последовательности выполняются задачи (поток управления), какие развилки и исключения возникают (шлюзы, события) и чем процесс завершается.

6. Оркестрация бизнес-процессов и отличие от автоматизации

Оркестрация — централизованное управление взаимодействием различных ИТ-систем, сервисов и людей в рамках единого сквозного процесса (как дирижёр управляет оркестром). Автоматизация — замена ручного труда машинным на уровне отдельной

изолированной задачи. Оркестрация координирует автоматизированные задачи между собой.

7. Различие оркестрации и хореографии

Оркестрация предполагает центральный «контроллер» (процесс в одном пуле), который отдаёт команды и управляет ходом выполнения. Хореография — децентрализованное взаимодействие: обмен сообщениями между независимыми пулами-партнёрами без единого руководителя, каждый участник действует по своим правилам.

8. «Исполняемый BPMN»

Это означает, что нарисованная бизнес-аналитиком диаграмма является не статичной картинкой, а транслируется в машиночитаемый код. BPMS-система (Camunda, Bizagi и др.) может «запустить» схему: назначать задачи людям, рассылать письма, вызывать скрипты и API согласно потоку управления.

9. Основные категории элементов нотации BPMN

– Узлы потока управления: события, задачи/действия, шлюзы.

– Соединяющие объекты: потоки управления, потоки сообщений, ассоциации.

– Зоны ответственности: пулы и дорожки (swimlanes).

– Артефакты: объекты данных, группы, текстовые аннотации.

10. Классификация событий. Промежуточные события

По расположению события делятся на стартовые, промежуточные и конечные. По триггеру — на обрабатывающие (catching: ждут наступления) и генерирующие (throwing: создают событие). К промежуточным относится всё, что происходит между стартом и концом и влияет на ход процесса: таймеры, получение/отправка сообщений, ошибки, сигналы, эскалации.

11. Классификация действий (Activities)

– Задача (Task) — атомарный, неделимый шаг процесса.

– Подпроцесс (Sub-process) — составное действие, внутри которого отдельная детализированная схема.

– Вызывающий процесс (Call Activity) — глобальный процесс, переиспользуемый в других моделях.

12. Классификация логических операторов (шлюзов)

– Эксклюзивный (XOR) — маркер «пусто» или «X»: выбирается одна ветка.

– Параллельный (AND) — маркер «+»: выполняются все ветки.

– Инклюзивный (OR) — маркер «O»: выполняются одна или несколько веток.

– Событийный (Event-based) — маркер-шестиугольник: выбор по наступившему событию.

– Комплексный (Complex) — маркер «*»: сложная логика, задаваемая условиями.

13. Классификация соединяющих объектов

– Поток управления (Sequence Flow) — сплошная линия со стрелкой, задаёт очередность.

– Поток сообщений (Message Flow) — штриховая линия с открытой стрелкой, связь между разными пулами.

– Ассоциация (Association) — пунктирная линия, связывает артефакты (текст, данные) с объектами.

14. Множественные экземпляры (Multi-instance)

Маркер задачи или подпроцесса (три горизонтальные или вертикальные черточки) означает, что действие выполняется многократно — по одному разу для каждого элемента списка (например, проверка каждого из 10 документов). Выполнение может быть параллельным или последовательным.

15. Ad-hoc-подпроцесс

Неструктурированный подпроцесс (маркер «тильда ~»): задачи внутри не соединены строгими потоками управления, исполнитель сам решает, какие задачи, в каком порядке и сколько раз выполнять для достижения цели.

16. Оператор для синхронизации

Это сходящийся шлюз (обычно параллельный AND или инклюзивный OR), который ждёт, пока токены не придут по всем активным входящим веткам, и только после этого объединяет их и пропускает процесс дальше.

17. Описание элементов на скриншоте

События: конечное генерирующее событие-сообщение, промежуточный таймер, промежуточное обрабатывающее событие-сообщение (получение), промежуточное генерирующее событие-сообщение (отправка), конечное событие-ошибка. Связи: поток по умолчанию, условный поток, поток сообщений, направленная ассоциация, поток управления.

18. Как отметить, что процесс продолжается только после обработки события

На поток управления помещается промежуточное обрабатывающее событие (catching event, например таймер или сообщение — двойная окружность со светлым значком). Токен доходит до него, останавливается и не идёт дальше, пока событие не наступит.

19. Промежуточное событие-таймер

Используется, чтобы приостановить процесс на заданное время («ждать 2 часа»), дождаться конкретной даты, либо — при прикреплении к границе задачи — прервать зависшую задачу по таймауту («если нет ответа 3 дня — отменить»).

20. Граничные события в BPMN

Это события, прикреплённые к границе задачи или подпроцесса; они обрабатывают ситуации, возникающие во время выполнения. Прерывающие (сплошной контур)

отменяют задачу; непрерывающие (штриховой контур) запускают параллельную ветку, не отменяя задачу. Примеры: граничный таймер, граничная ошибка.

21. Различные типы задач (Task) в BPMN

– Пользовательская (User Task) — выполняется человеком в интерфейсе ИТ-системы.

– Сервисная (Service Task) — автоматически системой/роботом (вызов API).

– Задача-скрипт (Script Task) — исполнение фрагмента кода движком.

– Задача отправки/получения (Send/Receive Task) — коммуникация сообщениями.

– Ручная (Manual Task) — физическая работа без компьютера.

– Бизнес-правило (Business Rule Task) — обращение к таблице принятия решений.

2. Лабораторная работа 6. Модель анализа

2.1. Контрольные вопросы

1. Что такое модель домена и с какой целью она создаётся?

Модель домена (модель предметной области, концептуальная схема классов) — упрощённая диаграмма классов, описывающая ключевые понятия предметной области, их атрибуты и отношения. Она создаётся на этапе анализа для формирования единого словаря терминов, понимания структуры предметной области, выявления связей и ограничений, а также как основа требований, вариантов использования, API и приёмочных тестов. Модель не содержит деталей реализации: операций, интерфейсов и структуры базы данных.

2. Поясните сценарий использования. Где и кем используется?

Сценарий варианта использования — текстовое описание последовательности взаимодействия актёра и системы для достижения цели. Он включает основной поток, альтернативные и исключительные потоки, пред- и постусловия. Сценарий используется аналитиками для фиксации требований, заказчиками — для согласования, разработчиками — для проектирования, тестировщиками — для создания тест-кейсов; на его основе строят диаграммы деятельности и последовательности.

3. Какие виды отношений используются в Use Case?

Ассоциация (актёр — вариант использования), включение (include), расширение (extend) и обобщение (между актёрами и между вариантами использования).

4. Как создаются альтернативные потоки?

Из основного потока выявляют отклонения: неверные данные, недоступность ресурса, таймаут, отказ, техническую ошибку, отмену. Для каждого отклонения указывают условие или событие-триггер, шаги, точку возврата в основной поток либо завершение и постусловия. Альтернативный поток не должен дублировать основной и должен быть проверяемым.

5. Назовите возможные варианты реализаций Use Case.

Текстовый сценарий; диаграмма деятельности; диаграмма последовательности (в том числе системная); диаграмма коммуникации; диаграмма состояний для поведения отдельного объекта; прототип пользовательского интерфейса; тест-кейсы и приёмочные тесты; непосредственно реализация в коде (кооперация).

6. Что такое точка расширения, условие активации и ветка расширения?

Точка расширения (extension point) — именованное место в базовом варианте использования, куда может быть вставлено расширение. Условие активации (activation condition) — логическое условие, при котором расширение выполняется. Ветка расширения (extension branch) — последовательность шагов расширяющего варианта,

вставляемая в точке расширения; отношение extend направлено от расширяющего варианта к базовому.

7. Какие другие элементы UML допустимо использовать на диаграмме UC, кроме основных?

Пакеты и подсистемы (границы системы), примечания и ограничения, зависимости, обобщения, отношения include/extend, точки расширения, артефакты (в инструментах), а также вложенные диаграммы деятельности или последовательности для детализации сценариев.

8. Назовите основные типы актёров.

Первичные (основные) и вторичные (вспомогательные); активные и пассивные; люди и внешние системы; абстрактные актёры. По роли: клиент, гид, руководитель, администратор, внешняя система.

9. Приведите пример обобщения действующих лиц (актёров).

Абстрактный актёр «Сотрудник бюро» обобщает актёров «Гид» и «Руководитель (Администратор)»: дочерние актёры наследуют его ассоциации и варианты использования, например «Авторизоваться в системе». Другой пример: «Пользователь» — «Клиент» и «Администратор».

10. Что такое абстрактное действующее лицо?

Актёр, не имеющий прямых экземпляров и задающий общее поведение или связи для своих специализаций; его имя обычно выделяют курсивом. Он используется для наследования вариантов использования и ассоциаций конкретными актёрами.

2.2. Контрольные вопросы по UML

1. Назовите типы сущностей в UML.

Структурные, поведенческие, группирующие и аннотационные сущности.

2. Что относится к структурным сущностям?

Класс, интерфейс, кооперация (collaboration), прецедент (use case), объект, компонент, артефакт, узел. В совокупности структурные сущности называют классификаторами; действующее лицо (actor) — разновидность классификатора.

3. Что относится к поведенческим сущностям?

Основные поведенческие сущности: состояние, деятельность (activity) и действие (action). К поведенческим также относят вариант использования, взаимодействие (interaction) и автомат (state machine).

4. Назовите семь наиболее важных свойств классификаторов.

1) Имя — идентифицирует элемент в пространстве имён. 2) Наличие экземпляров (прямых и косвенных). 3) Абстрактность или конкретность. 4) Видимость (public,

protected, private, package). 5) Область действия составляющих. 6) Кратность. 7) Только классификаторы могут участвовать в отношении обобщения.

5. Что в UML относится к классификаторам?

Классы, интерфейсы, типы данных, перечисления, действующие лица, варианты использования, компоненты, узлы, артефакты, кооперации, ассоциации. Объект — это экземпляр, а не классификатор.

6. Что такое абстрактный классификатор?

Классификатор, который не может иметь прямых экземпляров; его имя выделяется курсивом. Он описывает общие свойства и операции, а его экземплярами считаются экземпляры конкретных специализаций (например, интерфейс или абстрактный класс).

7. Что такое классификатор?

Дескриптор множества однотипных объектов, описывающий их структурные и поведенческие свойства (атрибуты, операции, отношения, ограничения) и способный иметь экземпляры.

8. Что такое дескриптор?

Описание общих свойств множества объектов: структуры, отношений, поведения, ограничений и назначения. Дескриптор имеет две стороны: intent (само описание множества) и extent (множество описываемых значений). Важнейший вид дескриптора — классификатор.

9. Что такое литерал? Чем отличается от дескриптора?

Литерал описывает сам себя — это конкретное значение (число 1, строка «abc», true). Дескриптор описывает множество объектов (например, тип integer описывает множество целых чисел). Литерал — значение, дескриптор — тип/классификатор.

10. Дайте понятия состояния, деятельности, действия.

Состояние — период жизненного цикла объекта, в котором он удовлетворяет условию, выполняет деятельность или ожидает события. Деятельность — продолжительные неатомарные вычисления (поток работ). Действие — примитивное атомарное вычисление, шаг деятельности.

11. Является ли сущность UC классификатором? Обоснуйте ответ.

Да. В UML вариант использования — поведенческий классификатор (BehavioredClassifier): он имеет имя, может иметь экземпляры (сценарии), участвовать в обобщении, включать точки расширения и отношения include/extend. При этом UC не является классом в узком смысле, но классификатором является.

12. Что относится к группирующим сущностям?

Пакеты (packages) — основная группирующая сущность; также модели, подсистемы и представления. Пакет может содержать структурные, поведенческие и другие группирующие элементы; он существует только на этапе разработки.

13. Что такое программный интерфейс? Что такое сигнатура?

Интерфейс — именованное множество операций (и, возможно, атрибутов/сигналов), определяющее контракт услуг, которые поставщик предоставляет потребителю; он не содержит реализации. Сигнатура — имя операции, список параметров (типы, направление, кратность) и тип возвращаемого значения (при необходимости — исключения); она однозначно идентифицирует операцию.

14. Дайте определение компонента UML.

Компонент — модульная, заменяемая часть системы с чётко определённым набором предоставляемых и требуемых интерфейсов, скрывающая реализацию и допускающая развёртывание.

15. Что такое артефакт? Приведите примеры.

Артефакт — физическая единица информации, используемая или порождаемая в процессе разработки и развёртывания. Примеры: файлы исходного кода, .class, .jar, .exe, .dll, SQL-скрипты, конфигурационные файлы, библиотеки, документы.

16. Какие типы отношений определены в UML?

Ассоциация (включая агрегацию и композицию), обобщение, реализация, зависимость. Частные виды зависимости: include, extend, использование, уточнение, трассировка; в диаграммах состояний — переход.

17. Назовите типы отношений в UML. Дайте им определения. Как они обозначаются?

Ассоциация — семантическая связь между классификаторами; изображается сплошной линией, при необходимости с кратностями и навигацией. Агрегация — «часть — целое» с независимым существованием частей; пустой ромб у целого. Композиция — сильная агрегация, части не существуют без целого; заполненный ромб. Обобщение — «является»; сплошная линия с пустым треугольником у родителя. Реализация — класс реализует интерфейс; пунктирная линия с пустым треугольником. Зависимость — изменение одного элемента влияет на другой; пунктирная стрелка. Include/extend — пунктирные стрелки с соответствующим стереотипом.

18. Назовите основные типы диаграмм. Дайте им краткую характеристику.

Классов — структура классов и отношений; объектов — экземпляры и связи; пакетов — группировка элементов; компонентов — компоненты и интерфейсы; развёртывания — узлы и артефакты; композитной структуры — внутреннее строение; вариантов использования — актёры, UC и отношения; деятельности — поток работ и действий; состояний — состояния и переходы объекта; последовательности — упорядоченные во времени взаимодействия; коммуникации — взаимодействия с акцентом на связи;

синхронизации (timing) — изменение состояний во времени; обзора взаимодействий — обзор нескольких сценариев.

19. Какие представления включает модель «4+1»?

Логическое (структура классов и объектов), процессное (процессы, потоки, производительность), разработки/реализации (организация кода и компонентов), физическое/развёртывания (узлы и размещение артефактов) и представление вариантов использования (сценарии, «+1»), связывающее остальные.

3. Лабораторная работа 7. Проектирование веб-API

1. Что еще, кроме REST API, можно использовать для организации связи между приложениями?

SOAP, GraphQL, gRPC, WebSocket, JSON-RPC/XML-RPC, брокеры сообщений (RabbitMQ, Kafka), webhooks, очереди задач, а также файловый обмен и общие базы данных.

2. Какие типы API существуют?

По назначению и аудитории: публичные, партнёрские, внутренние (private). По стилю и протоколу: REST, SOAP, GraphQL, gRPC, WebSocket, RPC. По уровню: библиотечные, операционные, веб-API.

3. Приведите примеры использования API.

Платёжные API (эквайринг, возвраты), картографические и геолокационные сервисы, погодные API, мессенджеры и SMS, CRM/ERP-интеграции, маркетплейсы и логистика, аутентификация через внешние сервисы (OAuth).

4. Что такое спецификация API?

Машинно-читаемый контракт API (например, OpenAPI/Swagger), описывающий серверы, эндпоинты, HTTP-методы, параметры, схемы запросов и ответов, аутентификацию и ошибки. Спецификация служит единым источником истины для документации, генерации кода и тестирования.

5. Что такое конечная точка API?

Конкретный адрес (URI) вместе с HTTP-методом, по которому клиент отправляет запрос и получает ответ. Конечная точка является частью маршрута ресурса; у одного ресурса может быть несколько конечных точек.

6. Какие задачи решаются на этапе предварительного проектирования API?

Определение ресурсов и их связей, назначение URL, выбор действий (HTTP-методов), определение входных параметров и результатов, проектирование структур данных, проверка полноты параметров, устранение влияния поставщика и привязка целей к UC.

7. Какие команды HTTP являются идемпотентными?

GET, HEAD, PUT, DELETE, OPTIONS, TRACE. Повторный вызов с теми же данными даёт тот же результат состояния. POST и PATCH в общем случае неидемпотентны; для платежей идемпотентность обеспечивается заголовком Idempotency-Key.

8. Существует ли клиентский веб-API?

Да. В браузере доступны клиентские Web API: Fetch, DOM, Web Storage, Geolocation, Notifications, Canvas и др. Они предоставляются браузером, а не сервером; серверный веб-API — это удалённый API, доступный по HTTP.

9. Из чего состоит серверный веб-API?

Из набора конечных точек и маршрутов, HTTP-методов, форматов запросов и ответов (JSON/XML), аутентификации и авторизации, бизнес-логики, слоя доступа к данным, обработки ошибок, журналирования и документации (спецификации).

10. Как в REST API передаются данные на сервер?

Через параметры пути (/contracts/{contractId}), строку запроса (GET ?search=…&page=…), заголовки (Authorization, X-Client-Token, Idempotency-Key) и тело запроса (JSON для POST/PUT/PATCH, multipart/form-data для файлов).

11. Из каких шагов складывается обработка API-запросов?

Маршрутизация запроса, аутентификация и проверка прав, валидация параметров и тела, выполнение бизнес-логики, обращение к данным и внешним сервисам, формирование ответа и кода состояния, журналирование и возврат ответа клиенту.

12. Продолжите предложение «При разработке API важно иметь глубокие и точные знания о том, …»

…кто может использовать API; что он может делать; как он это делает; что ему нужно для этого; что он получает взамен. Эти сведения образуют основу целей API и позволяют спроектировать интерфейс с точки зрения потребителя.

4. Лабораторная работа 8. Документирование веб-API

1. Что означает «API-first подход»?

Подход, при котором сначала разрабатывается и фиксируется контракт API (спецификация), а затем реализуется сервер и клиенты. При расхождении контракта и реализации изменяют реализацию, а не контракт.

2. Назовите типы веб-API.

REST, SOAP, GraphQL, gRPC, WebSocket, JSON-RPC/XML-RPC; по аудитории — публичные, партнёрские и внутренние (private); по способу применения — клиентские и серверные.

3. Что означает слово REST?

Representational State Transfer — «передача состояния представления»; архитектурный стиль взаимодействия компонентов распределённой системы.

4. Что такое веб-сервис? Какой тип API использует веб-сервис?

Веб-сервис — программная служба, доступная по сети через стандартизированный интерфейс. Чаще всего используется REST (RESTful веб-сервис) или SOAP.

5. Какие функции выполняет API-шлюз?

Маршрутизацию запросов, аутентификацию и авторизацию, ограничение скорости, кэширование, трансляцию протоколов, агрегацию ответов, логирование и мониторинг, управление версиями и балансировку нагрузки.

6. Почему REST не является протоколом?

REST — набор архитектурных ограничений (стиль), а не протокол передачи данных. Он не определяет формат сообщений и транспорт; обычно используется HTTP, но сам стиль может применяться и с другими протоколами.

7. Почему архитектура, основанная на событиях, не будет удовлетворять требованиям RESTful?

Событийная архитектура асинхронна и основана на публикации/подписке, а REST предполагает клиент-инициированные запросы-ответы, отсутствие состояния на сервере и единообразный интерфейс. Постоянные соединения и push-уведомления нарушают stateless-модель и единообразие интерфейса.

8. Назовите основные принципы REST.

Клиент-сервер, отсутствие состояния (stateless), кэшируемость, единообразный интерфейс, многоуровневая система, код по требованию (необязательно).

9. Какую цель преследует требование «RESTful-система должна производить операции в клиент-серверной модели»?

Разделение ответственности: клиент отвечает за пользовательский интерфейс, сервер — за данные и бизнес-логику. Это позволяет развивать части независимо и повышает масштабируемость.

10. В чем смысл требования «Отсутствие состояния» в RESTful-системе?

Сервер не хранит состояние сессии клиента между запросами: каждый запрос содержит всю необходимую информацию (токен, параметры). Это упрощает масштабирование, восстановление после сбоев и кэширование.

11. Принцип унификации или единообразия интерфейса. Как достигается унификация?

За счёт единых правил идентификации ресурсов (URI), использования стандартных HTTP-методов и кодов состояния, единых форматов представления (JSON/XML) и гипермедиа-ссылок.

12. Назовите четыре составляющих унифицированного интерфейса.

Идентификация ресурсов; манипулирование ресурсами через представления; самоописываемые сообщения; гипермедиа как движок состояния приложения (HATEOAS).

13. Как реализуется унификация интерфейса в REST?

Ресурсы адресуются URI, действия выполняются стандартными HTTP-методами (GET, POST, PUT, PATCH, DELETE), ответы используют единые коды состояния и медиа-типы, а связи между ресурсами выражаются гиперссылками.

14. Почему возникла необходимость разработки ключа API? В чем недостатки базовой аутентификации?

Ключ API позволяет идентифицировать клиентское приложение, ограничивать доступ и квоты, не передавая логин и пароль. Базовая аутентификация передаёт учётные данные в каждом запросе (base64), не даёт гранулярных прав, сложнее в ротации и при компрометации пароля требует его смены.

15. Есть ли альтернативы REST?

Да: GraphQL, gRPC, SOAP, WebSocket, JSON-RPC/XML-RPC, событийные архитектуры и брокеры сообщений (Kafka, RabbitMQ), а также файловый обмен и очереди задач.